

FOX学习心得

最近在使用 ajax 做小b商家项目，在项目开发的过程也对对框架本身产生了浓厚的兴趣，尤其是 FOX 框架，它和我们主流的开发框架有什么区别？（如react）。下面是我最近一段时间的学习心得。从几个问题入手去寻找答案。

- 1.为什么存在 FOX 框架？
- 2.它解决了那些问题？
- 3.它是怎样提供了一个开发模型来约束视图、数据、逻辑？

不得不说的 AJAX 框架发展史

第一代框架 ajaxFramework。有幸还能在官网找到这个一代框架，虽然已经被废弃，但也是理解 FOX 产生原因的一个重要角度。阅读文档不难发现，ajaxFramework 虽然是使用 jsx 作为组件，但还是它对 DOM 的操作依旧是传统的命令式。下面是官网提供的 Demo。修改属性需要找到对应节点执行 setAttribute。

[复制代码](#)

```
class MyComponent extends AJAX.Component {
  // 声明初始视图
  getInitialView() {
    return (
      <div ref="container" className="empty">
        {/* 1.给需要操作的DOM元素绑定ref */}
        <label ref="content" text="加载中..."></label>
      </div>
    );
  }
  // 请求数据，更新视图
  componentDidMount() {
    // 1.通过ref拿到DOM元素
    let container = this.refs.container;
    let loadingTip = this.refs.content;
    // 2.操作DOM
    // （请求接口fetch在开发中，这里异步模拟）
    ajax.setTimeout(function() {
      // 拿到数据
      let data = '已经是最新了';
```

```
// 更新DOM
container.className = 'nonEmpty';
loadingTip.setAttribute('text', data);
}, 600);
}
```

这样的逻辑与UI混合在一起，除了维护性差开发效率低之外，还有其他问题比如说无生态可用问题等等，那么在React、Vue、Angular 已经遍布业界的时候为什么会出现ajaxFramework 框架呢？通过一个相关资料的学习了解到它是 ajax 业务发展的过渡产物，解决的核心问题是 UI 渲染。基建和业务同时开展的情况下很难一步到位。面对大量的需求迭代，FOX 框架出现，采用 react DSL，框架继承 React 设计理念，结合dva，向主流框架迈进。

选择 Rax

FOX 如何解决接入主流框架这一问题？首先为什么 vue 和 react 不能直接运行在 ajax 上面，是因为 DOM API 的差异导致，以 react 为例，理论上只需要参考 react-dom，写一套适合 ajax 的渲染库接口。实际上，FOX 选择了 Rax，一个使用 react DSL 的跨端框架，Rax 的 vdom 和调度分离的非常好，API去实现driver的抽象即可。可以更大程度上节约开发成本。我们看下 rax 文档。只需要实现 dirver 相关规范，就能轻松实现跨平台。

✉️**Write Once, Run Anywhere:** write one codebase, run with `Web` , `Weex` , `Node.js` , `Alibaba MiniApp` , `WeChat MiniProgram` and could work with more container that implements [driver specification](#) .

下面是 rax-app 的部分代码。raxRender 和 driver 相互分离。FOX 只需写 FOX-driver 即可。

▼ 复制代码

```
import { render as raxRender } from 'rax';
import DriverUniversal from 'driver-universal';
function mount(appInstance, rootEl) {
  return app = raxRender(appInstance, rootEl, {
    driver: DriverUniversal
  });
}
```

FOX driver

下面是截取的 FOX driver 源码，——实现 getElementById、createBody、createEmpty、createText...等17 个API。

▼ 复制代码

```
var Driver = {
```

```

    getElementsByById: function getElementsByById(id) {
        return ajax.ui.getElementsByById(id);
    },
    createElement: function createElement(type, props, _internal) {
        var vNode = createVNodeFactory(type, props, _internal);
        this.setNativeProps(vNode, props, false);
        return vNode;
    }
}
...

```

如果有兴趣可以详细看下 Driver 中的每个 API 实现这里不做展开，就是在 Driver 的实现过程中，FOX 封装了一层 vNode，在通过 vNode 再去操作 ajax.ui。通过一些资料的学习我了解到 vNode 是为了解决实现中的一些问题。那么下面我通过一些源码的学习去理解 FOX 是如何解决这些问题的。

- AJX渲染引擎cell的子、孙节点是不能被修改的
- AJX渲染引擎的createElement+append的性能要远远低于innerHTML，所以必须要用tree的方式整体拼装好
- AJX无法用一个节点作为Empty节点，不同的tag下需要用不同的Empty节点

.....

先回忆下 list 组件 中的 cell 组件。在嵌套规则中 list 组件下只能是 section 标签，而section下面只能是cell 或者header或者footer标签。



复制代码

```

<list>
  <section>
    <cell>
      <label text="test"><label>
    </cell>
    <cell></cell>
  </section>
</list>

```

下面是 cell 组件官方文档中的注意事项。

注意

- cell组件无法应用margin、width、position相关样式（不会报错）。盒模型样式只支持padding、height、border。
- 已经添加到页面上的cell组件（可从 ajax.ui.root 遍历到），其子元素除可修改样式（如color、padding等）及属性（如text等）外，不可移除（removeChild）、添加（appendChild）、替换（replaceChild）、插入（insertBefore）等操作。如有业务需要，以上操作可通过修改元素的display样式来实现。
- 还未添加到页面文档中的 cell（不可从 ajax.ui.root 遍历到）可以通过 appendChild、removeChild 操作子节点

重点就是第二条，cell 元素的子元素不可移除，不可替换，不可插入。但是需要更新 cell 组件的子节点的怎么办？如果使用 display 可能不能满足全部的需求场景。那么最直接有效的方式，就是把 cell 更新后的节点以及子节点封装成新的节点树，一次性替换掉已经添加到页面上的 cell 元素。

那么回到之前的问题，即使需要一次性替换 cell 节点树，为什么要封装 vNode，我觉得如果在driver 中直接操作 ajax.ui 可能会存在以下问题。如果 cell 下面的任何一个子节点发生变动，如果被移除或者被替换，那么都会导致 cell 整个节点树都会被更新。那么可能会浪费很多不必要的性能。打个比方，某个cell元素先插入一个子节点然后在更新一个子节点，正确的解法应该是记录变动然后拼装成一个新的节点树，一次性替换 cell 及其子节点。而不是每做一次变动替换一次。

那么这种情况怎样解决？有两中解决方案，第一是去修改 Rax 源代码，在 diff patch 过程中添加对 cell 元素的处理，但是对 rax 入侵性太大，不利于后续的生态接入。第二种方式是封装 vNode，对操作 cell 的节点及其子节点在 vNode 中进行 isInCell 标识，当操作 cell 节点及其子节点的时候不出处理，只是放到待更新列表中，等待 RAX committed阶段再一次性处理。

实现 VNodeFactory

既然可以通过封装 vNode 解决 cell 的问题，那么我们来看下源码的实现。

1.首先，实现一个 vNode 工厂， 里面有个 isInCell 和 nearlyCell，在每个元素创建的时候，就去判断是否为 cell 元素或者其子节点然后标识为 isInCell。再把子元素 nearlyCell 赋值为最近的 cell 元素。方便后续读取这个属性更新所有子节点。

▼

复制代码

```
function createVNodeFactory(type, props, _internal) {
  var vNode = {
    nodeType: ELEMENT_NODE,
    tagName: type,
    isPure: true,
    isVNode: true,
    x: uuid++,
    isInCell: false,
    nearlyCell: null,
    ...
  }
  ...
}
```

2.在 createVNodeFactory 的 appendChild、removeChild 等方法,添加 对 cell 元素的判断，如果为 cell 或者其子元素则不做 appendChild 处理。只是修改了 vNode 然后把 nearlyCell 放进了 pending\$1 中。pending\$1 的后续操作先按下不说，我们先看下 isInCell 和 nearlyCell 是怎样判断的

▼

复制代码

```
function appendChild(vNode) {
  if (this.isInCell && this._internal) {
```

```

    this._internal._changed = true;
}

this.childNodes.push(vNode);
vNode.parentNode = this;

if (this.isInCell || this.tagName === 'cell') {
    var changeCell = this.nearlyCell;

    if (changeCell && pending$.indexOf(changeCell) < 0) {
        pending$.push(changeCell);
    }

    return;
}

if (!this.isPure) {
    this.getNativeNode().appendChild(vNode.getNativeNode());
}
}

```

3.判断 `isInCell` 和 `nearlyCell`，在首次渲染的时候，根节点的 `vNode` 会执行 `getNativeNode` 的方式，把 `vNode` 转化成 真实的 native 节点，下面我们来看下代码实现



复制代码

```

function getNativeNode() {
    var node = null;

    if (this._nativeNode) {
        return this._nativeNode;
    } else if (!this.isPure) {
        node = ajax.ui.getElementByX(this.x);
        this._nativeNode = node;
    } else {
        var xmlStr = xmlHelper.createXML(this);
        var _node2 = null;

        try {
            _node2 = ajax.ui.inflate(xmlStr);
        } catch (e) {
            xmlStr = xmlStr.replace(/\u0000/g, '');
            _node2 = ajax.ui.inflate(xmlStr);
        }

        this._nativeNode = _node2;
    }
}

```

```
    if (!this._nativeNode) ;

    return this._nativeNode;
  },
```

内部使用了 `xmlHelper.createXML` 方法

▼ 复制代码

```
var xmlHelper = {
  createXML: function createXML(rootVNode) {
    rootVNode.isPure = false;
    var xml = '<' + rootVNode.tagName + ' '; // x

    xml += UNI_ID_KEY + '=' + '"' + rootVNode.x + '" '; // attrStr

    xml += rootVNode.getAttrStr();
    xml += '>'; // children

    var innerXml = '';
    rootVNode.childNodes.forEach(function (childVNode) {
      childVNode._nativeNode = null;

      if (rootVNode.tagName === 'cell') {
        rootVNode.nearlyCell = rootVNode;
        childVNode.nearlyCell = rootVNode;
        childVNode.isInCell = true;
      }

      if (rootVNode.isInCell) {
        childVNode.isInCell = true;
        childVNode.nearlyCell = rootVNode.nearlyCell;
      }

      innerXml += xmlHelper.createXML(childVNode);
    });
    xml += innerXml;
    xml += '</' + rootVNode.tagName + '>';
    return xml;
  }
}
```

简单来说，就是在首次渲染的时候吧 `vNode` 递归解析合成字符串，并通过 `ajax.ui.inflate` 的方式一次性生成 `native` 节点。至于用 `inflate` 而不是 `createElement`，我觉得可能是性能上更好一些。在解析 `vnode` 的过程中判断 `isInCell` 和 `nearlyCell`。

3.放在 pending\$1 中的 cell 文件什么时候更新呢？继续观察 FOX 的实现。

在源码中我找到了下面的代码，通过 while 循环调用 replaceChild 更新 整个 cell 节点。在代码中，我们发现 makeLifecycleBeforeReplace 和 makeLifecycleAfterReplace，这里做了生命周期补偿，就是通过 下面的 collectInstance 方法，收集所有 cell 节点最近的Component实例，然后调用其 componentWillUnmount 和 componentDidMount方法。这里不做详细代码展示。

▽ 复制代码

```
function flush$1() {
  var cellObj = pending$1.shift();

  while (cellObj) {
    var cell = cellObj;
    var x = cell.x;
    cell._nativeNode = null;
    cell.isPure = true;
    var newCell = cell.getNativeNode();
    var oldCell = ajax.ui.getElementByX(x);
    collectInstance(cell);

    if (oldCell && oldCell.parentNode) {
      makeLifecycleBeforeReplace();
      oldCell.parentNode.replaceChild(newCell, oldCell);
      makeLifecycleAfterReplace();
    }

    cellObj = pending$1.shift();
  }
}
```

最后在 RAX 中的 performUpdate 中调用 flush\$1 更新cell 。

处理 createEmpty

list 子元素的用法，同样影响了对 driver api 中createEmpty 的处理。

- list下只能是section标签
- section下只能是cell标签或者header或者footer标签
- cell标签下可以正常嵌套div,img,label等标签

▽ 复制代码

```
function createEmpty(that) {
```

```

var type = 'div';
var parentInstance = that.__parentInstance;

while (parentInstance) {
  if (parentInstance && typeof parentInstance.type === 'string') {
    if (parentInstance.type === 'section') {
      type = 'cell';
    }

    if (parentInstance.type === 'list') {
      type = 'section';
    }

    break;
  } else {
    parentInstance = parentInstance._internal.__parentInstance;
  }
}

return this.createElement(type, {
  style: {
    display: "none"
  }
});
}

```

createEmpty 方法中需要判断当前元素的最近父元素是否为 list 或者 section，从而对空元素做处理。

整合dva

就像 react 只是 DOM 的一个抽象层一样，只有 RAX 并不是 web 应用的整体解决方案，对于大型的复杂应用来说，还需要代码结构和组件间的通信，dva 就很好的解决了这两个问题。它整合了 redux，redux-saga 的数据流方案。FOX 把dva 整合进框架中，并删除了dva中对于 redux-saga 的整合。原因是在是 redux-saga在AMap下的IO和执行时间在Iphone 5s上几乎要花掉80ms(参考 [高德前端框架思考与展望](#) 一文)。换成了轻量的异步中间件。我们先看下源码，FOX 使用了什么方法来实现了 saga 的基础功能。

首先实现了写了 bllMiddleware 来代替 saga 中间件。createBllMiddleware 方法中实现了异步中间件的所有处理。



复制代码

```

var middleware = createBllMiddleware();
middleware.withExtraArgument = createBllMiddleware;
var bllMiddleware = middleware;

```

```

var store = app._store = createStore$1({
  reducers: createReducer(),
  initialState: hooksAndOpts.initialState || {},
  plugin: plugin,
  bllMiddleware: bllMiddleware,
  createOpts: createOpts
});

```

createBllMiddleware 中主要是实现了 put, call, select。

▼
复制代码

```

var fnArg2 = {
  put: function put(action) {
    if (action && action.type && action.type.indexOf('/')>0) {
      return dispatch(Object.assign({}, action));
    } else {
      var newaction = Object.assign({}, action, { type:
effectMaps[curNamespace].namespace + NAMESPACE_SEP + action.type });
      return dispatch(newaction);
    }
  },
  call: function call(fn) {
    for (var _len = arguments.length, args = Array(_len > 1 ? _len - 1
: 0), _key = 1; _key < _len; _key++) {
      args[_key - 1] = arguments[_key];
    }

    return fn.apply(fn, args);
  },
  select: function select(fn) {
    if (typeof fn === 'function') {
      return fn(getState());
    } else {
      return getState();
    }
  }
};

```

然后调用runGenerator，递归调用 next 这里不再展开。

page 实现

dva 是怎样在项目中是使用的呢？如何 connect 组件以及我们如何使用 dispatch。还有page中生命周期的实现。下面一起来通过源码来看看 FOX 的实现。

model 的引用

首先看下 dva 是如何引用的，在下面代码中可以看到在 FOXPage 的 constructor 中引入了 dva，并且把 dispatch 放入 props 对象中。在 constructor 最下方调用 _initModel 注册 model。

▽ 复制代码

```
export default class FOXPage extends Lifecycle{
  constructor(container) {
    super(container)
    let self = this;
    self.rootNode = container || ajax.ui.root;
    self.props = {};

    self.appRoot = dva();
    self.appRoot.createStore();
    self.props.dispatch = self.appRoot.getDispatch();

    AJAX.dispatch = self.props.dispatch;
    AJAX.addModel = this.model.bind(this);

    self._initStyle();
    self._initModel();
  }
  ....
}
....
_initModel(){
  let modelArr = this.registerModel();
  modelArr && modelArr.forEach((model)=>{
    this.appRoot.model(models);
  })
}
```

初始化 model 之后，会调用 App.start(CommodityPageName); 这里会先初始化 ajax 生命周期，等到 ready 之后会创建 Container AJAX 组件，已经调用 react-redux connect 方法

▽ 复制代码

```
function mapStateToProps(state) {
  return self.state(state);
}
self.rootView = connect(mapStateToProps)(Container)
```

生命周期

page 的生命周期是包含了 AJAX 周期和 Rax 生命周期，在代码中可以清晰看到 page 中的 RAX 生命周期是映射 Container 组件的生命周期

[复制代码](#)

```
class Container extends AJAX.Component{
  constructor(props){
    super(props);
    props.dispatch = self.props.dispatch;
  }
  componentWillMount(){
    self.componentWillMount && self.componentWillMount();
  }
  ....
}
```

而 AJAX 的生命周期则是通过 ajax 生命周期 `ajax.addListener` 监听获得。进而通过返回值触发其他生命周期，如 `onActive` 等。

[复制代码](#)

```
ajax.addListener('pageshow', (appSwitch, param)=> {...}
....

LifeCycleManager.addEvent(LifeCycleEventsType.PAGE_SHOW, (appSwitch,
param)=> {
  //接收navigator.back传过来的参数
  if (param) {
    self.pageShow(appSwitch, param);
  } else {
    self.pageShow(appSwitch);
  }
  if (!self.loaded && typeof self.onActive === 'function') {
    self.onActive(param);
    self.loaded = true;
  } else if (appSwitch && typeof self.onForeground === 'function') {
    self.onForeground(param);
  } else if (!appSwitch && typeof self.onResume === 'function') {
    self.onResume(param);
  }
});
```

总结

这里一共从三个方面分享最近对FOX 的学习分享。分别为如何接入RAX，如何整合dva和如何实现page。部分参考文档，其他大部分主要是对源码的理解。解决了心里的很多疑惑对FOX 有了更深入的了解。在学习的过程中，感受到了团队的在框架探索中的不懈努力，相信未来会越来越完善，更好的开发体验，更快的性能。

贡献

[高德前端框架思考与展望](#)

[AJX教程](#)